

```

// Package main demonstrates comprehensive spreadsheet creation with ML results
data,
// including model comparisons, formulas, and charts.
//
// nolint:revive // Example file demonstrating comprehensive spreadsheet features
package main

import (
    "fmt"
    "log"

    "github.com/connerohnesorge/goffice/spreadsheet"
)

const (
    // Row offsets for data placement in different sheets
    modelDataStartRow      = 5
    analysisDataStartRow   = 4
    trainingDataStartRow   = 4
    hyperparamDataStartRow = 4
)

// ML Paper Results Example creates a complex spreadsheet with:
// - Model comparison data with multiple metrics
// - Formulas for computed columns (improvement %, speedup, etc.)
// - Multiple charts (line, bar, scatter)
// - Professional styling and formatting
// - Demonstrates comprehensive spreadsheet capabilities
func main() {
    if err := run(); err != nil {
        log.Fatalf("Error: %v", err)
    }
}

func run() error {
    // Create a new document
    doc, err := spreadsheet.Create(
        "ml_results_example.xlsx",
        spreadsheet.DocTypeWorkbook,
    )
    if err != nil {
        return fmt.Errorf(
            "creating document: %w",
            err,
        )
    }
    defer func() {
        if err := doc.Close(); err != nil {
            log.Printf(
                "Error closing document: %v",
                err,
            )
        }
    }()

    // Create all sheets
    if err := createModelResultsSheet(doc); err != nil {
        return fmt.Errorf(
            "creating model results sheet: %w",

```

```

        err,
    )
}

if err := createAnalysisSheet(doc); err != nil {
    return fmt.Errorf(
        "creating analysis sheet: %w",
        err,
    )
}

if err := createTrainingSheet(doc); err != nil {
    return fmt.Errorf(
        "creating training sheet: %w",
        err,
    )
}

if err := createHyperparameterSheet(doc); err != nil {
    return fmt.Errorf(
        "creating hyperparameter sheet: %w",
        err,
    )
}

// Save the document
if err := doc.SaveAs("ml_results_example.xlsx"); err != nil {
    return fmt.Errorf(
        "saving document: %w",
        err,
    )
}

printSummary()

return nil
}

//
//nolint:revive // Function length acceptable for sheet creation with data
//population
func createModelResultsSheet(
    doc *spreadsheet.Document,
) error {
    // ===== Sheet 1: Model Results =====
    sheet, err := doc.AddSheet("Model Results")
    if err != nil {
        return err
    }

    // Title and headers
    sheet.Cell("A1").
        SetString("Deep Learning Model Performance Comparison")
    sheet.Cell("A2").
        SetString("Neural Network Architecture Benchmark")

    // Add some spacing
    sheet.Cell("A4").SetString("Model Name")
    sheet.Cell("B4").SetString("Parameters (M)")

```

```

sheet.Cell("C4").
    SetString("Training Time (hrs)")
sheet.Cell("D4").
    SetString("Inference Speed (imgs/sec)")
sheet.Cell("E4").
    SetString("ImageNet Top-1 Acc %")
sheet.Cell("F4").
    SetString("ImageNet Top-5 Acc %")
sheet.Cell("G4").SetString("COCO mAP")
sheet.Cell("H4").SetString("Memory (GB)")

// Model data rows
models := getModelData()
for i, model := range models {
    row := modelDataStartRow + i
    sheet.Cell(fmt.Sprintf("A%d", row)).
        SetString(model.name)
    sheet.Cell(fmt.Sprintf("B%d", row)).
        SetNumber(model.params)
    sheet.Cell(fmt.Sprintf("C%d", row)).
        SetNumber(model.train)
    sheet.Cell(fmt.Sprintf("D%d", row)).
        SetNumber(model.speed)
    sheet.Cell(fmt.Sprintf("E%d", row)).
        SetNumber(model.top1)
    sheet.Cell(fmt.Sprintf("F%d", row)).
        SetNumber(model.top5)
    sheet.Cell(fmt.Sprintf("G%d", row)).
        SetNumber(model.mAP)
    sheet.Cell(fmt.Sprintf("H%d", row)).
        SetNumber(model.memory)
}

// Chart 1: Model Accuracy Comparison (Bar Chart)
if len(models) > 0 {
    chart1, err := sheet.AddChart(
        spreadsheet.ChartTypeBar,
        "E5:E14",
        spreadsheet.CellRef{Row: 15, Col: 1},
    )
    if err != nil {
        log.Printf(
            "Warning: Could not create accuracy comparison chart:
%v\n",
            err,
        )
    } else {
        chart1.SetTitle("Model Accuracy Comparison (ImageNet Top-1 %)")
        chart1.AddSeries("Accuracy %", "E5:E14")
    }
}

return nil
}

//
//nolint:revive // Function length acceptable for sheet creation with formulas
func createAnalysisSheet(
    doc *spreadsheet.Document,

```

```

) error {
    // ===== Sheet 2: Computed Analysis =====
    analysisSheet, err := doc.AddSheet("Analysis")
    if err != nil {
        return err
    }

    // Headers for analysis
    analysisSheet.Cell("A1").
        SetString("Model Performance Analysis")
    analysisSheet.Cell("A3").
        SetString("Model Name")
    analysisSheet.Cell("B3").
        SetString("Params (M)")
    analysisSheet.Cell("C3").
        SetString("Accuracy (Top-1 %)")
    analysisSheet.Cell("D3").
        SetString("Efficiency Score")
    analysisSheet.Cell("E3").
        SetString("Params-per-Accuracy")
    analysisSheet.Cell("F3").
        SetString("Speed Ranking")
    analysisSheet.Cell("G3").
        SetString("Accuracy Gain vs ResNet-50")
    analysisSheet.Cell("H3").
        SetString("Memory per Accuracy")

    // Data for analysis with formulas
    analysisData := getAnalysisData()
    for i, data := range analysisData {
        row := analysisDataStartRow + i
        analysisSheet.Cell(fmt.Sprintf("A%d", row)).
            SetString(data.name)
        analysisSheet.Cell(fmt.Sprintf("B%d", row)).
            SetNumber(data.params)
        analysisSheet.Cell(fmt.Sprintf("C%d", row)).
            SetNumber(data.acc)

        // Efficiency Score = Accuracy / Parameters (higher is better)
        efficiencyFormula := fmt.Sprintf(
            "=C%d/B%d",
            row,
            row,
        )
        analysisSheet.Cell(fmt.Sprintf("D%d", row)).
            SetFormula(efficiencyFormula)

        // Params per Accuracy point
        paramsPerAccFormula := fmt.Sprintf(
            "=B%d/(C%d-50)",
            row,
            row,
        )
        analysisSheet.Cell(fmt.Sprintf("E%d", row)).
            SetFormula(paramsPerAccFormula)

        // Accuracy gain vs ResNet-50
        gainFormula := fmt.Sprintf(
            "=C%d-$C$4",

```

```

        row,
    )
    analysisSheet.Cell(fmt.Sprintf("G%d", row)).
        SetFormula(gainFormula)

    // Memory per accuracy
    memoryPerAccFormula := fmt.Sprintf(
        "=B%d*1.5/(C%d-50)",
        row,
        row,
    )
    analysisSheet.Cell(fmt.Sprintf("H%d", row)).
        SetFormula(memoryPerAccFormula)
}

// Chart 3: Parameters vs Accuracy Scatter
if len(analysisData) > 0 {
    chart3, err := analysisSheet.AddChart(
        spreadsheet.ChartTypeScatter,
        "B4:B13",
        spreadsheet.CellRef{Row: 15, Col: 1},
    )
    if err != nil {
        log.Printf(
            "Warning: Could not create scatter chart: %v\n",
            err,
        )
    } else {
        chart3.SetTitle("Model Parameters vs Accuracy Trade-off")
        chart3.AddSeries("Accuracy", "C4:C13")
    }
}

return nil
}

//
//nolint:revive // Function length acceptable for sheet creation with data
population
func createTrainingSheet(
    doc *spreadsheet.Document,
) error {
    // ===== Sheet 3: Training Metrics Over Time =====
    trainingSheet, err := doc.AddSheet(
        "Training Curve",
    )
    if err != nil {
        return err
    }

    trainingSheet.Cell("A1").
        SetString("Training Progress - Loss and Accuracy")
    trainingSheet.Cell("A3").SetString("Epoch")
    trainingSheet.Cell("B3").
        SetString("Training Loss")
    trainingSheet.Cell("C3").
        SetString("Validation Loss")
    trainingSheet.Cell("D3").
        SetString("Training Accuracy %")

```

```

trainingSheet.Cell("E3").
    SetString("Validation Accuracy %")
trainingSheet.Cell("F3").
    SetString("Learning Rate")
trainingSheet.Cell("G3").
    SetString("Overfitting Gap (Val-Train)")

// Simulated training data
epochs := getEpochData()
for i, e := range epochs {
    row := trainingDataStartRow + i
    trainingSheet.Cell(fmt.Sprintf("A%d", row)).
        SetNumber(float64(e.epoch))
    trainingSheet.Cell(fmt.Sprintf("B%d", row)).
        SetNumber(e.trLoss)
    trainingSheet.Cell(fmt.Sprintf("C%d", row)).
        SetNumber(e.valLoss)
    trainingSheet.Cell(fmt.Sprintf("D%d", row)).
        SetNumber(e.trAcc)
    trainingSheet.Cell(fmt.Sprintf("E%d", row)).
        SetNumber(e.valAcc)
    trainingSheet.Cell(fmt.Sprintf("F%d", row)).
        SetNumber(e.lr)

    // Add formula for gap between training and validation loss
    gapFormula := fmt.Sprintf(
        "=C%d-B%d",
        row,
        row,
    )
    trainingSheet.Cell(fmt.Sprintf("G%d", row)).
        SetFormula(gapFormula)
}

// Chart 2: Training Curve (Line Chart)
if len(epochs) > 0 {
    chart2, err := trainingSheet.AddChart(
        spreadsheet.ChartTypeLine,
        "D4:E12",
        spreadsheet.CellRef{Row: 15, Col: 1},
    )
    if err != nil {
        log.Printf(
            "Warning: Could not create training curve chart: %v\n",
            err,
        )
    } else {
        chart2.SetTitle("Training vs Validation Accuracy Over Epochs")
        chart2.AddSeries("Training Acc", "D4:D12")
        chart2.AddSeries("Validation Acc", "E4:E12")
    }
}

return nil
}

func createHyperparameterSheet(
    doc *spreadsheet.Document,
) error {

```

```

// ===== Sheet 4: Hyperparameter Search =====
hpSheet, err := doc.AddSheet(
    "Hyperparameter Search",
)
if err != nil {
    return err
}

hpSheet.Cell("A1").
    SetString("Hyperparameter Tuning Results")
hpSheet.Cell("A3").SetString("Experiment")
hpSheet.Cell("B3").SetString("Learning Rate")
hpSheet.Cell("C3").SetString("Batch Size")
hpSheet.Cell("D3").SetString("Weight Decay")
hpSheet.Cell("E3").SetString("Warmup Steps")
hpSheet.Cell("F3").
    SetString("Final Accuracy %")
hpSheet.Cell("G3").
    SetString("Convergence Time (hrs)")
hpSheet.Cell("H3").
    SetString("Accuracy per Hour")

hpExperiments := getHyperparameterData()
for i, hp := range hpExperiments {
    row := hyperparamDataStartRow + i
    hpSheet.Cell(fmt.Sprintf("A%d", row)).
        SetNumber(float64(hp.exp))
    hpSheet.Cell(fmt.Sprintf("B%d", row)).
        SetNumber(hp.lr)
    hpSheet.Cell(fmt.Sprintf("C%d", row)).
        SetNumber(float64(hp.batch))
    hpSheet.Cell(fmt.Sprintf("D%d", row)).
        SetNumber(hp.wd)
    hpSheet.Cell(fmt.Sprintf("E%d", row)).
        SetNumber(float64(hp.warmup))
    hpSheet.Cell(fmt.Sprintf("F%d", row)).
        SetNumber(hp.acc)
    hpSheet.Cell(fmt.Sprintf("G%d", row)).
        SetNumber(hp.time)

    // Accuracy per hour = Accuracy / Time
    accPerHourFormula := fmt.Sprintf(
        "=F%d/G%d",
        row,
        row,
    )
    hpSheet.Cell(fmt.Sprintf("H%d", row)).
        SetFormula(accPerHourFormula)
}

return nil
}

func printSummary() {
    fmt.Println(
        "Successfully created ml_results_example.xlsx",
    )
    fmt.Println("\nSpreadsheet contains:")
    fmt.Println(

```

```

        " - Sheet 1 'Model Results': 10 model comparisons with 8 performance
metrics",
    )
    fmt.Println(
        " - Sheet 2 'Analysis': Computed columns with formulas (efficiency,
gains, etc.)",
    )
    fmt.Println(
        " - Sheet 3 'Training Curve': 9 epochs of training data with
loss/accuracy curves",
    )
    fmt.Println(
        " - Sheet 4 'Hyperparameter Search': 8 hyperparameter experiments with
efficiency metrics",
    )
    fmt.Println(
        " - Multiple charts: Bar chart (models), Line chart (training),
Scatter (params vs accuracy)",
    )
    fmt.Println(
        " - Formula-based computed columns demonstrating mathematical
operations",
    )
    fmt.Println("\nTest with LibreOffice:")
    fmt.Println(
        " soffice --headless --convert-to pdf ml_results_example.xlsx",
    )
    fmt.Println(
        " soffice --headless --calc ml_results_example.xlsx",
    )
}

```